Classmate... No.

MCS - 211

Design and Analysis of Algorithm

Q-1.a) Develop an efficient algorithm to find a list of prime numbers from 100 to 1000.

=) one common and efficient algorithm to find prime numbers within a given range is the Sieve of Eratosthenes.

Here's a Python implementation to find prime numbers between 100 and 1000.

```
def sieve of - eratosthenes (start, end):

    # Create a boolan array "prime[0-
    end]" and initialize all entries

    # value in prime[i] will be false
      if i is not a prime, and
      True or prime = [true for- in
      range(and +1)]

    p= 2

    while p** 2 <= end:

    # If prime[p] is not changed,
      then it is a prime.

      if prime[p]:

      # update all multiples of p.

        for i in range(p** 2, end+1,
            p):
          prime[i] = False
      p += 1
```

```
# Collect all prime numbers in
  the specified range.

primes = [i for i in range (max (2,
          start), end + 1) if prime
          [i]]
return primes

# Find prime numbers between
  100 and 1000.

start, range = 100

end - range = 1000

result = sieve_of_eratosthenes (
          start, range, end - range)

# Print the list of prime numbers:

print (result).
```

- This algorithm works by iteratively marking the multiples of each prime numbers.

(b) Differentiate between Polynomial-time and expotential-time algorithms, Give an example of one problem each for these two running times.

=) Polynomial-time and exponential-time are classification of algorithms based on their running time in terms of the input size.

1. ** Polynomial-time algorithms: **

- ** Definitions: **

An algorithms is said to run in polynomial time if its running time is bounded by a polynomial function of the input size.

In other words, if the running time is $O(n^K)$ for some constant K, where n is the input size, then it's a polynomial-time algorithm.

- ** Example Problem: **

Polynomial - time algorithms are generally considered efficient. An example problem that can be solved in polynomial time is finding the shortest path between two nodes in a graph algorithms like Dijkstra's algorithms or the Floyd - Warshall algorithms.

2. ** Exponential - time algorithms: **

** Definitions: **

An algorithms is said to run is exponential time if its running time is proportional to some exponential function of the input size.

In other words, if the running time is $O(a^n)$ for some constant a, where n is the input size.

(c) Using Horner's rule, evaluate the polynomial $p(x) = 2x^5 - 5x^4 - 3x^2 + 15$ at $x = 2$.
Analyse the computation time required for polynomial evaluation using Horner's rule against the Brute force method.

Ans- Horner's rule is an efficient method for evaluating polynomials. It is based on the idea of nested multiplication and can be used to evaluate polynomials at a given value of x much faster than the standards method for expanding the polynomials and simplifying.

Here are the steps involved in using Horner's rule to evaluate the polynomial $p(x) = 2x^5 - 5x^4 - 3x^2 + 15$ at $x = 2$:

1. write down the coefficients of the polynomial term.

Classmate No.

Q-2.(a) What is a Greedy Approach to Problem-solving? Formulate the fractional knapsack problem as an optimisation problem and write a greedy algorithm to solve this problem. Solve the following fractional knapsack problem using this algorithm.
Show all the steps.
Suppose there is a knapsack of capacity 15 kg and 6 items are to packed in it.
The weight and profit of the items are as under:

$(p_1, p_2, \ldots, p_6) = (3, 2, 4, 5, 1, 6)$

$(w_1, w_2 \ldots, w_6) \quad (2, 1, 2, 1, 5, 1)$

Ans Greedy Algorithms for Fractional knapsack Problem:

1. Calculate the profit-to-weight ratio for each item:

Divide the profit of each item by

its weight to get the profit-to-weight ratio.

2. Sort the items in decreasing order of profit-to-weight ratio:

This ensures that we are picking the items that provide the most profit per unit of weight first.

3. Start filling the knapsack with the items in sorted order:

- If the item fits entirely in the knapsack, add it completely.
- If the items doesn't fit entirely, add a fraction of it that fills the remaining knapsack space exactly.

4. Calculate the profit:

Sum the profits of all the items added to the knapsack.

Classmate... No.

item	Profit (p-i)	weight (wi)	pi-wi
1	3	2	1.5
2	2	1	2.0
3	4	2	2.0
4	5	1	5.0
5	1	5	0.2
6	6	1	6.0

1. Sort the items in decreasing order of p-i / w-i : { 6, 4, 2, 3, 2, 1 }

2. Start filling the knapsack:

- Item 6:
  It firstly, perfectly (weight = 1, remaining capacity = 15).

- Item 4:
  If fits perfectly (weight = 1, remaining capacity = 14).
- Item 2:
  It doesn't fit. Add 1 unit (weight = 1, remaining capacity = 13).
- Item 3:
  It doesn't fit. Add 1 unit (weight = 1, remaining capacity = 12)
- Item 1:
  It doesn't fit skip it
- Item 5:
  It doesn't fit. Skip it.

3. Total profit:
   $6 + 4 + 1 + 1 = 12$

Therefore, the maximum profit that can be achieved with the given knapsack capacity is 12.

b) What is the purpose of using Huffman codes? Explain the steps of building a huffman tree. Design the Huffman codes for the following set of characteristics and their frequencies:

a: 15, e: 19, s: 5, d: 6, f: 4, g: 7, h: 8, t: 10.

Ans- ** Purpose of Huffman Codes: **

Huffman coding is a popular algorithm used for lossless data compression. The primary purpose of Huffman codes is to represent data in a more compact form by assigning shorter codes to more frequent symbols and longer codes to less.

This results in efficient compression, especially for data with varying symbol frequencies.

Classmate... No.

** Steps to Build a Huffman Tree: **

1. ** Frequency Table: **

Create a frequency table for each symbol in the given set along with its frequency.

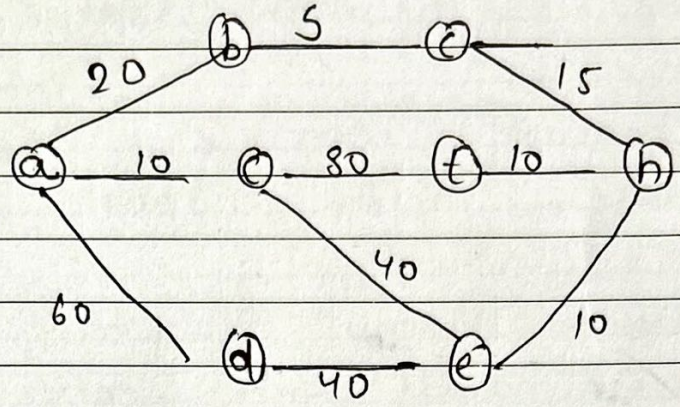
Symbol	Frequency
a	15
e	19
s	5
d	6
f	4
g	7
h	8
t	10

2. ** create Nodes: **

Create a leaf node for each symbol with its frequency.

Classmate... No.

Q-3 Consider the following graph:



(a) Write the Kruskal's algorithm and Prim's algorithm to find the minimum cost spanning tree of the graph given in figure 1. Show all the steps of computation. Also compute the time complexity of both the algorithms.

Ans- Let's start with Kruskal's algorithm and then move on to Prim's algorithm.

Kruskal's Algorithm:

1. Sort all the edges in non-decreasing order of their weight.

2. Pick the smallest edge. check if it forms a cycle with the spanning tree formed so far.

If cycle is not formed, include this edge. Else, discard it.

3. Repeat step 2 until there are (V-1) edges in the spanning tree.

The time complexity of this algorithm is O(ElogE) or O(ElogV). Sorting of edges takes O(ElogE) time.

After sorting, we iterate through all edges and apply the find-union algorithm. The find and union operation can take at most O(LogV) time.

So, overall complexity is O(ElogE + ElogV) time. The value of E can be at most O(V^2), so, O(LogV) and O(LogE) are the

the same. Therefore, the overall time complexity is $O(E \log E)$ or $(E \log v)$.

Prim's Algorithm:-

1. Initialize a tree with a single vertex, choosen arbitraily from the graph.

Grow the tree by one edge:
of the edges that connect the tree to vertices not yet in the tree, find the minimum-weight edge, and transfer it to the tree.

3. Repeat step 2 (until all vertices are in the tree).

The time complexity of Prim's algorithm depends on the data structures used for the graph and for ordering the edges by weight.

(b) what is dynamic programming? what is the principle of optimality? Use the dynamic programming approach to find the optimal sequences of chain multiplication of the following matrices:

Matrix	Dimensions
A1	5 x 10
A2	10 x 20
A3	20 x 15
A4	15 x 8
A5	8 x 10

Ans- let's solve this problem using dynamic programming.

Dynamic Programming:-

It is a method for solving complex problems by breaking them down into simpler subproblems.

It is used when the subproblems are not independent. i.e, when the solution to one subproblems may involve solutions to other subproblems.

The Principle of optimality states that an optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.

Now, let's find the optimal sequence of chain multiplication for the given matrices using dynamic programming.

The matrices are as follows:

- A1 = 5X10
- A2 : 10X20

A3: 20 x 15

A4 : 15 x 8

A5 : 8 x 10

The dynamic programming for matrix chain multiplication is as follows:

(d) Make all the possible Binary Search trees for the key values 25, 50, 75.

Ans- Let's construct all possible Binary Search Trees (BSTs) for the key 25, 50, and 75.

1

50 /
2575

2

25
50
75

Q4(a) what are decision problems and optimission problems? Differentiate the decision problems and optimisation problems with the help of at least two problems statement of each.

Ans- ** Decision Problems: **

Decision problems are a type of computational problem where the answer is either "yes" or "no" based on some criteria or conditions.

The goal is to decide whether a given input satisfies a certain property or constant. Decision problems are often fundamental in theoretical computer science.

It can be classified into various complexity classes.

** Examples of Decision Problems: **

1. **Subset Sum:**

Input:

A set of positive integers S and a target integer T.

Question:

Does there exist a subset of S such that the sum of its elements equals T?

2. **Graph Reachability:**

- Input:

A directed graph G, and two graph u and v.

- Question:-

Is there a path from vertex u to vertex v in the graph G?

(b) Define P and NP class of Problems with the help of examples. How are P class of problem different from NP class of Problems

Ans- ** P and NP Classes: **

The classes P and NP are classifications of decisions problems in computational complexity theory.

1. ** P(Polynomial Time): **

- A decision problem is in the class P if there exists an algorithm that can solve it in polynomial time.

- An algorithm is considered efficient if its running time is polynomial in the size of the input.

** Example of a P Problem: **

- ** Problem: **

Classmate... No.

Sorting a list of number.

- ** Explanation: **

Algorithms like Quicksort or Merge sort can sort a list of n numbers in O(n log n) time, which is polynomial in the size of the input.

2. ** NP (Nondeterministic Polynomial Time): **

- A decision problem post is in the class NP if, given a proposed solution, it can be verified as correct in polynomial time.

- NP stands- for-

"nondeterministic polynomial" because it involves a non-deterministic polynomial-time verifier.

(c) what are NP- Hard and NP- complete problem? what is the role of reduction? Explain with the help of an example.

Ans- ** NP- Hard and NP- complete Problems **

1. ** NP- Hard :**

- A problem is NP- hard (nondeterministic polynomial - time hard) if every problem in NP. can be reduced to it in polynomial time.

- NP- hard problems are at least as hard as the hardest problems in NP but may not be in NP.

2. ** NP- Complete :**

- A problem is NP- complete if it is both in NP and NP - hard.

- NP- complete problems are a subset of NP- hard problems that are

believed to be the hardest problems in NP.

** Role of Reduction: **

Reduction is a technique used to establish the hardness of problems by transforming one problem into another in a way that preserves the computational complexity.

Specifically, polynomial-time reductions are used, meaning the transformation can be performed in polynomial time.

** Example- Reduction: **

let's consider two problems: "Subset Sum" and "3SAT".

1. "Subset Sum": **

- Problem:-

Given a set of integers and a target

d) Define the following Problems:

(i) 3- CNF SAT

(ii) Clique problem

(iii) vertex cover problem

(iv) Graph Colouring Problem

Ans(i) ** i) 3- CNF SAT (3- Cluase Normal form Satisfiability: **

** Problem Definition: **

Given a Boolean formula in 3- CNF (3- clause Normal Form), where each clause consists of the disjunction (OR) of exactly three literals on their negations.

The CNF SAT Problems asks whether there exists an assignment of truth values to the variables that satisfies the entire formula.